

NAME: M.Greeshma

COURSE: Design and Analysis of Algorithms

CODE: CSA0619

REPORT

23. Analyze Binary Search performance when executed repeatedly on streaming data with periodic sorting. Record execution time for both sorting and searching phases. Interpret trade-offs between preprocessing cost and search efficiency. Justify the use of Binary Search in dynamic data streams.

Sol..

Code:

```
import random
```

```
import time
```

```
import bisect
```

```
def binary_search(arr, target):
```

```
    """Binary search using bisect module for simplicity."""
```

```
    idx = bisect.bisect_left(arr, target)
```

```
if idx < len(arr) and arr[idx] == target:
```

```
    return idx
```

```
return -1
```

```
# Simulation parameters
```

```
stream_size = 10000    # total elements in stream
```

```
batch_size = 1000      # sort after every batch
```

```
search_trials = 100    # number of searches per batch
```

```
stream_data = []
```

```
sorting_times = []
```

```
searching_times = []
```

```
for i in range(0, stream_size, batch_size):
```

```
    # Simulate incoming batch of data
```

```
    batch = [random.randint(1, 100000) for _ in range(batch_size)]
```

```
    stream_data.extend(batch)
```

```
    # Sorting phase
```

```
    start_sort = time.time()
```

```
    stream_data.sort()
```

```
    end_sort = time.time()
```

```
    sorting_times.append(end_sort - start_sort)
```

```
# Searching phase
```

```
start_search = time.time()
```

```
for _ in range(search_trials):
```

```
    target = random.choice(stream_data)
```

```
    binary_search(stream_data, target)
```

```
end_search = time.time()
```

```
searching_times.append(end_search - start_search)
```

```
# Results
```

```
for i, (s_time, q_time) in enumerate(zip(sorting_times,  
searching_times), 1):
```

```
    print(f"Batch {i}: Sorting time = {s_time:.6f}s, Searching time =  
{q_time:.6f}s")
```

```
print("\nAverage Sorting Time:",  
sum(sorting_times)/len(sorting_times))
```

```
print("Average Searching Time:",  
sum(searching_times)/len(searching_times))
```

```
main.py
1 import random
2 import time
3 import bisect
4 def binary_search(arr, target):
5     """Binary search using bisect module for simplicity."""
6     idx = bisect.bisect_left(arr, target)
7     if idx < len(arr) and arr[idx] == target:
8         return idx
9     return -1
10 stream_size = 10000      # total elements in stream
11 batch_size = 1000        # sort after every batch
12 search_trials = 100      # number of searches per batch
13 stream_data = []
14 sorting_times = []
15 searching_times = []
16 for i in range(0, stream_size, batch_size):
17     # Simulate incoming batch of data
18     batch = [random.randint(1, 100000) for _ in range(batch_size)]
19     stream_data.extend(batch)

Batch 1: Sorting time = 0.000130s, Searching time = 0.000150s
Batch 2: Sorting time = 0.000168s, Searching time = 0.000154s
Batch 3: Sorting time = 0.000185s, Searching time = 0.000158s
Batch 4: Sorting time = 0.000187s, Searching time = 0.000166s
Batch 5: Sorting time = 0.000202s, Searching time = 0.000156s
Batch 6: Sorting time = 0.000299s, Searching time = 0.000164s
Batch 7: Sorting time = 0.000234s, Searching time = 0.000164s
Batch 8: Sorting time = 0.000237s, Searching time = 0.000153s
Batch 9: Sorting time = 0.000276s, Searching time = 0.000179s
Batch 10: Sorting time = 0.000308s, Searching time = 0.000169s

Average Sorting Time: 0.0002227306365966797
Average Searching Time: 0.00016126632690429687

...Program finished with exit code 0
Press ENTER to exit console.
```

Performance Analysis

1. Sorting Phase (Preprocessing Cost)

- Sorting is required to maintain the order of incoming streaming data.
- Common algorithms:
 - Merge Sort / Quick Sort $\rightarrow O(n \log n)$

- **Insertion Sort (incremental updates)** $\rightarrow O(n^2)$ worst case, but efficient if data is nearly sorted.

- Sorting dominates the preprocessing cost, especially when the stream is large and updates are frequent.

2. Searching Phase (Binary Search Efficiency)

- Once sorted, Binary Search operates in $O(\log n)$.
- For repeated queries, this logarithmic cost is negligible compared to sorting.
- Example: For $n = 1,000,000$, sorting may take millions of operations, while Binary Search needs only ~ 20 comparisons.

Execution Time Recording

- **Sorting Time:** increases with stream size and frequency of re-sorting.
- **Search Time:** remains consistently low due to logarithmic complexity.
- **Combined Cost:**
 - Frequent sorting $\rightarrow$ high overhead.
 - Infrequent sorting $\rightarrow$ lower overhead but risk of outdated results.

Trade-offs

- **High Preprocessing Cost vs. Fast Queries:**
 - Sorting is expensive but amortized over multiple searches.
 - If queries are frequent, the upfront cost of sorting is justified.

- **Dynamic Streams:**
 - Continuous updates require either:
 - **Periodic full sorting** (batch updates).
 - **Incremental maintenance** (balanced trees, heaps, or self-adjusting structures).
- **Latency vs. Accuracy:**
 - More frequent sorting → lower query latency, higher accuracy.
 - Less frequent sorting → reduced preprocessing, but queries may miss recent updates.

Justification for Binary Search in Dynamic Streams

- **Efficiency:** Binary Search ensures queries are answered quickly, critical in real-time systems.
- **Scalability:** Handles millions of records with minimal query time.
- **Balance:** Even though sorting is costly, amortizing it across many queries makes Binary Search optimal compared to linear search.
- **Alternatives:** Data structures like balanced BSTs, B-trees, or skip lists can reduce the need for repeated sorting, but Binary Search remains the simplest and most predictable method when periodic sorting is acceptable.

Conclusion

Binary Search is justified in dynamic data streams because:

- Sorting cost is a one-time or periodic overhead.

- Query efficiency ($O(\log n)$) drastically improves responsiveness.
- The trade-off favors Binary Search when query frequency is high relative to update frequency.